K-Nearest Neighbours and Approximate Search

ES 335 Machine Learning, Semester I 2025–26 Sources: lecture "K-nearest neighbours", cheat sheet ML22, and the curse-of-dimensionality and KNN-variants notebooks

What this file is really about:

$$\text{store the data} \rightarrow \text{measure distance} \rightarrow \text{vote among the nearest} \rightarrow \text{choose } K \rightarrow \text{search faster}$$

How to use this file. Part A builds the method. Part B shows the code. Part C is practice with answers.

Step-up path

You need. Distance between points and the idea of a held-out set.

1. Predict from the closest stored examples.
2. Choose a distance and scale the features.
3. Choose K on held-out data.
4. See why high dimensions make distances blur.
5. Search faster with trees and hashes.

You can stop when you can pick K honestly and explain when KNN is the wrong tool.

Contents

Part A Maths From the Ground Up

A1 Similar inputs, similar outputs

The whole method rests on one assumption: points close together tend to have similar answers. So keep every training example, and for a new query q:

1. Compute its distance to every stored example.
2. Take the K closest.
3. Predict their majority class, or the mean of their values for regression.

With $K = 1$ each training point owns a **Voronoi cell**, the region closer to it than to any other point. The boundary between two cells is a piece of a perpendicular bisector, so the decision boundary is piecewise linear.

A2 Lazy and non-parametric

	KNN	Linear regression
Training	store the table	solve for θ
Each prediction	N distances, $O(ND)$	one dot product, $O(D)$
Memory	all $N \times D$ numbers	$D + 1$ numbers
Assumes	near means similar	a linear form

KNN is **lazy** because the work happens at prediction time. It is **non-parametric** because what it keeps grows with the data. These are separate ideas: a decision tree is non-parametric, but it is eager.

A3 Choosing a distance

The family of L_p distances is

$$d_p(a, b) = \left(\sum_j |a_j - b_j|^p \right)^{1/p}.$$

$p = 1$ is Manhattan, $p = 2$ is Euclidean, and $p = \infty$ is the largest gap over all coordinates. For binary features, Hamming distance counts the bits that differ.

Worked example. From $q = (7, 4)$, the point $(9, 6)$ is at L_1 distance 4, L_2 distance 2.83, and L_∞ distance 2. Its rank among neighbours depends on the choice.

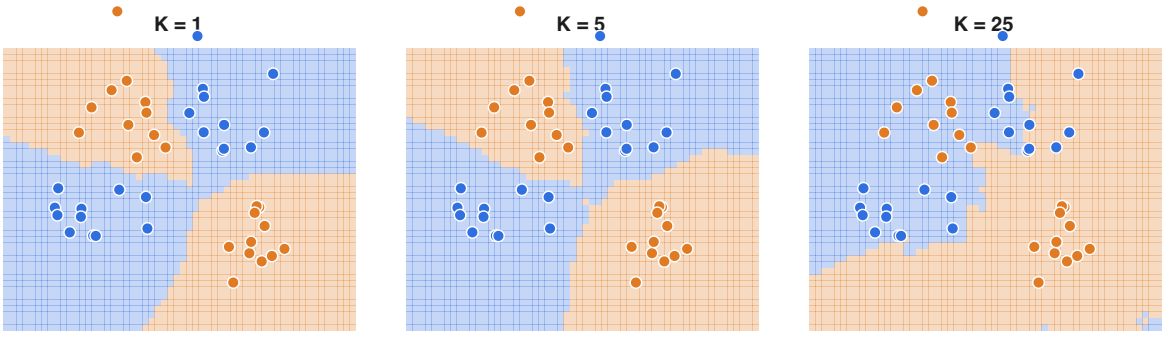
A4 Scale before you measure

Careful. Distances add across features, so the feature with the largest numbers dominates. A fruit's redness runs from 0 to 1, and its weight in grams runs into hundreds. Weight then decides who is near, even if it carries no information. Standardise using statistics from the training set only.

A5 Choosing K

Small K follows noise: at $K = 1$ training error is zero, because each point is its own nearest neighbour. Large K smooths too much: at $K = n$ every query gets the overall majority.

So K trades variance against bias, and training error cannot choose it. Use cross-validation, and an odd K for two classes.



The same points classified with three values of K. Small K creates islands around single points, while large K gives a smooth boundary.

A6 Weighted votes

Closer neighbours can count more, using weights such as $1/d$. A vote can flip: two red neighbours at distances 0.5 and 0.8 lose a plain vote to three blue neighbours at 1.5, 1.7 and 2.0, but win by weight,

since $2 + 1.25 = 3.25$ beats $0.67 + 0.59 + 0.5 = 1.76$.

A7 The curse of dimensionality

As dimensions grow, all points become about equally far apart. With ten uniform points, the ratio of nearest to farthest distance rises from about 0.08 in one dimension to 0.57 in ten and 0.85 in a hundred.

Volume also behaves strangely. A share $1 - 0.9^d$ of a ball lies in its outer 10% shell, and a cube holding 10% of uniform data has edge $0.1^{1/d}$, which is 0.79 at $d = 10$. A "neighbourhood" then spans most of every axis.

A8 Searching faster

Brute force costs $O(ND)$ per query. Two ideas do some work up front so that a query looks at only a subset.

kd-tree. Split the points at the median of one coordinate, then the next, alternating. A query walks down to one leaf. This is fast in low dimensions, but it can miss a true neighbour just across a boundary unless neighbouring boxes are also checked.

Locality-sensitive hashing. Draw P random hyperplanes. Each point gets a code from which side of each plane it lies on, and near points tend to share a code. Two vectors at angle θ agree on one bit with probability $1 - \theta/\pi$, and on all P with $(1 - \theta/\pi)^P$. Using several tables raises the chance of finding the true neighbours.

Part B In Code

B1 The algorithm in three lines

```
def knn(X, y, q, k):
    d = np.linalg.norm(X - q, axis=1)
    idx = np.argsort(d)[:k]
    return np.bincount(y[idx]).argmax()
```

B2 Scaling correctly

```
scaler = StandardScaler().fit(X_train)   # fit on training data only
Xtr, Xte = scaler.transform(X_train), scaler.transform(X_test)
```

B3 Random hyperplane hashing

```
planes = rng.normal(size=(P, d))
codes = (X @ planes.T > 0)     # one row of bits per point
```

The notebooks **Variants of nearest neighbours** and **The curse of dimensionality** in this section run these on real data.

Part C Practice

P1. Find the L_1, L_2 and L_∞ distances from $(0, 0)$ to $(3, 4)$.

7, 5 and 4.

P2. Point A is at $(3, 0)$ and B at $(2, 2)$. Which is nearer to the origin under L_2, and under L_1?

Under L_2: 3 against 2.83, so B. Under L_1: 3 against 4, so A.

P3. Why is training error useless for choosing K?

At $K = 1$ each training point is its own neighbour, so training error is zero regardless of how good the model is.

P4. How much of a ball lies in the outer 10% shell at $d = 20$?

$1 - 0.9^{20} \approx 0.878$.

P5. $N = 10^6$, $D = 100$. How many subtraction-style operations does brute force take per query?

About $N \times D = 10^8$.

P6. Is a decision tree lazy?

No. It does its work at training time, although it is non-parametric.